Fast Computation of Multivariate Subresultants

Yunus GÜRLEK, Frederic XIA

Supervisor: Weija WANG

May 21, 2026

Outline

Introduction

Problem

Given $f, g \in R[x]$ with $\deg(f) = p > q = \deg(g)$, compute

$$\gcd(f, g), \qquad \mathrm{Res}(f, g), \qquad \left(\mathrm{sResP}_i(f, g)\right)_{0 \le i \le q}.$$

Introduction

Problem

Given $f, g \in R[x]$ with $\deg(f) = p > q = \deg(g)$, compute

$$\gcd(f,g), \qquad \mathrm{Res}(f,g), \qquad \left(\mathrm{sResP}_i(f,g)\right)_{0 \le i \le q}.$$

Main idea

Use subresultants and specialization to replace expensive symbolic computations by many cheaper univariate computations.

Introduction

Problem

Given $f, g \in R[x]$ with $\deg(f) = p > q = \deg(g)$, compute

$$\gcd(f, g), \qquad \mathrm{Res}(f, g), \qquad \left(\mathrm{sResP}_i(f, g)\right)_{0 \le i \le q}.$$

Main idea

Use subresultants and specialization to replace expensive symbolic computations by many cheaper univariate computations.

Background

Univariate and multivariate polynomials, GCD, resultants, subresultants and subresultant polynomials.

Euclidean Algorithm: Correct but Costly

Euclidean GCD:

$$r_0 = f, \qquad r_1 = g, \qquad r_{i+1} = r_{i-1} \bmod r_i.$$

Euclidean Algorithm: Correct but Costly

Euclidean GCD:

$$r_0 = f, \qquad r_1 = g, \qquad r_{i+1} = r_{i-1} \bmod r_i.$$

Issue

Over rings such as $\mathbb{Z}$, coefficient sizes may grow quickly during division.

Euclidean Algorithm: Correct but Costly

Euclidean GCD:

$$r_0 = f, \qquad r_1 = g, \qquad r_{i+1} = r_{i-1} \bmod r_i.$$

Issue

Over rings such as $\mathbb{Z}$, coefficient sizes may grow quickly during division.

$$f = x^8 + x^6 - 3x^4 - 3x^3 + 8x^2 + 2x - 5,$$
$$g = 3x^6 + 5x^4 - 4x^2 - 9x + 21.$$

$$r_2 = -\frac{5}{9}x^4 + \frac{1}{9}x^2 - \frac{1}{3}, \qquad r_3 = -\frac{117}{25}x^2 - 9x + \frac{441}{25},$$
$$r_4 = \frac{233150}{19773}x - \frac{102500}{6591}, \qquad r_5 = -\frac{1288744821}{543589225}.$$

$$\gcd(f, g) = 1.$$

Euclidean Algorithm: Correct but Costly

Euclidean GCD:

$$r_0 = f, \qquad r_1 = g, \qquad r_{i+1} = r_{i-1} \bmod r_i.$$

Issue

Over rings such as $\mathbb{Z}$, coefficient sizes may grow quickly during division.

$$f = x^8 + x^6 - 3x^4 - 3x^3 + 8x^2 + 2x - 5,$$
$$g = 3x^6 + 5x^4 - 4x^2 - 9x + 21.$$

$$r_2 = -\frac{5}{9}x^4 + \frac{1}{9}x^2 - \frac{1}{3}, \qquad r_3 = -\frac{117}{25}x^2 - 9x + \frac{441}{25},$$
$$r_4 = \frac{233150}{19773}x - \frac{102500}{6591}, \qquad r_5 = -\frac{1288744821}{543589225}.$$

$$\gcd(f, g) = 1.$$

Using Subresultants:

Key property

The subresultant subsequence of polynomials is proportional to the Euclidean remainder sequence, and GCD is the final non-zero subresultant polynomial.

Euclidean Algorithm: Correct but Costly

Euclidean GCD:

$$r_0 = f, \qquad r_1 = g, \qquad r_{i+1} = r_{i-1} \bmod r_i.$$

Issue

Over rings such as $\mathbb{Z}$, coefficient sizes may grow quickly during division.

$$f = x^8 + x^6 - 3x^4 - 3x^3 + 8x^2 + 2x - 5,$$
$$g = 3x^6 + 5x^4 - 4x^2 - 9x + 21.$$

$$r_2 = -\frac{5}{9}x^4 + \frac{1}{9}x^2 - \frac{1}{3}, \qquad r_3 = -\frac{117}{25}x^2 - 9x + \frac{441}{25},$$
$$r_4 = \frac{233150}{19773}x - \frac{102500}{6591}, \qquad r_5 = -\frac{1288744821}{543589225}.$$

$$\gcd(f, g) = 1.$$

Using Subresultants:

Key property

The subresultant subsequence of polynomials is proportional to the Euclidean remainder sequence, and GCD is the final non-zero subresultant polynomial.

For the same f and g:

$$\mathrm{sResP}_5(f, g) = 15x^4 - 3x^2 + 9,$$
$$\mathrm{sResP}_3(f, g) = 65x^2 + 125x - 245,$$
$$\mathrm{sResP}_2(f, g) = 9326x - 12300,$$
$$\mathrm{sResP}_0(f, g) = 260708.$$

$$\gcd(f, g) = 1.$$

State of the Art and Contribution

State of the art:

- 1750, Subresultants were already known to Euler.

State of the Art and Contribution

State of the art:

- 1750, Subresultants were already known to Euler.
- 1960, Collins used them heavily in computational algebra.

State of the Art and Contribution

State of the art:

- 1750, Subresultants were already known to Euler.
- 1960, Collins used them heavily in computational algebra.
- 2000, Lazard and Ducos improved the subresultant algorithm.

State of the Art and Contribution

State of the art:

- 1750, Subresultants were already known to Euler.
- 1960, Collins used them heavily in computational algebra.
- 2000, Lazard and Ducos improved the subresultant algorithm.
- today, FLINT and Maple use subresultant-based methods for resultants.
- today, Wolfram exposes `SubresultantPolynomials` directly.

State of the Art and Contribution

State of the art:

- 1750, Subresultants were already known to Euler.
- 1960, Collins used them heavily in computational algebra.
- 2000, Lazard and Ducos improved the subresultant algorithm.
- today, FLINT and Maple use subresultant-based methods for resultants.
- today, Wolfram exposes `SubresultantPolynomials` directly.

This work

Design and implement (on FLINT) a fast algorithm for subresultants over $\mathbb{Q}$, $\mathbb{Z}$, and $\mathbb{F}_p$.

State of the Art and Contribution

State of the art:

- 1750, Subresultants were already known to Euler.
- 1960, Collins used them heavily in computational algebra.
- 2000, Lazard and Ducos improved the subresultant algorithm.
- today, FLINT and Maple use subresultant-based methods for resultants.
- today, Wolfram exposes `SubresultantPolynomials` directly.

This work

Design and implement (on FLINT) a fast algorithm for subresultants over $\mathbb{Q}$, $\mathbb{Z}$, and $\mathbb{F}_p$.

Technique

Use evoluation and interpolation, generalizing Collins's resultant algorithm.

State of the Art and Contribution

State of the art:

- 1750, Subresultants were already known to Euler.
- 1960, Collins used them heavily in computational algebra.
- 2000, Lazard and Ducos improved the subresultant algorithm.
- today, FLINT and Maple use subresultant-based methods for resultants.
- today, Wolfram exposes `SubresultantPolynomials` directly.

This work

Design and implement (on FLINT) a fast algorithm for subresultants over $\mathbb{Q}$, $\mathbb{Z}$, and $\mathbb{F}_p$.

Technique

Use evoluation and interpolation, generalizing Collins's resultant algorithm.

Observed performance

Speedups up to $7\times$ on benchmarks with total degree of bitsize 6 and coefficient bit size 50, both on FMPZ and FMPQ.

Subresultant Polynomials

For $0 \leq i \leq q$ and $0 \leq j \leq i$, let $M_{i,j}(f, g)$ be the corresponding Sylvester-Habicht minor.

Definition

$$\mathrm{sResP}_i(f, g) = \sum_{j=0}^{i} \det(M_{i,j}(f, g))x^j.$$

Non-defective case

$$\deg\big(\mathrm{sResP}_i(f, g)\big) = i.$$

Subresultant Polynomials

For $0 \leq i \leq q$ and $0 \leq j \leq i$, let $M_{i,j}(f, g)$ be the corresponding Sylvester-Habicht minor.

Definition

$$\mathrm{sResP}_i(f, g) = \sum_{j=0}^{i} \det(M_{i,j}(f, g)) x^j.$$

Non-defective case

$$\deg\big(\mathrm{sResP}_i(f, g)\big) = i.$$

- Non-zero subresultants encode Euclidean remainders.
- Defective subresultants reveal degree drops.
- Zero subresultants indicate gaps in the sequence.
- Last non-zero subresultant polynomial equals the GCD.

Consequence

The GCD and resultant can be computed without division in the fraction field.

Multivariate Setting

Let

$$f, g \in \mathbb{Z}[x_1, \ldots, x_n].$$

View them as univariate polynomials in x_1:

$$f, g \in R[x_1], \qquad R = \mathbb{Z}[x_2, \ldots, x_n].$$

Goal

Compute

$$\mathrm{Res}_{x_1}(f, g), \qquad \mathrm{sResP}_i(f, g).$$

Multivariate Setting

Let

$$f, g \in \mathbb{Z}[x_1, \ldots, x_n].$$

View them as univariate polynomials in x_1:

$$f, g \in R[x_1], \qquad R = \mathbb{Z}[x_2, \ldots, x_n].$$

Goal

Compute

$$\mathrm{Res}_{x_1}(f, g), \qquad \mathrm{sResP}_i(f, g).$$

Symbolic computation over

$$R = \mathbb{Z}[x_2, \ldots, x_n]$$

can be expensive.

Instead, choose evaluation points

$$\eta \in \mathbb{Z}^{n-1}$$

and compute in

$$\mathbb{Z}[x_1].$$

Backbone of the Approach

Specialization of the Resultant
Let

$$h = \mathrm{lc}_{x_1}(f)\,\mathrm{lc}_{x_1}(g).$$

Bad points

Degree drops after evaluation can break the equality.

Backbone of the Approach

Specialization of the Resultant
Let

$$h = \mathrm{lc}_{x_1}(f)\,\mathrm{lc}_{x_1}(g).$$

Bad points

Degree drops after evaluation can break the equality.

Valid evaluation point

If $h(\eta) \neq 0$, then

$$\mathrm{Res}_{x_1}(f(\eta), g(\eta)) = \mathrm{Res}_{x_1}(f, g)(\eta).$$

Backbone of the Approach

Specialization of the Resultant
Let

$$h = \mathrm{lc}_{x_1}(f)\,\mathrm{lc}_{x_1}(g).$$

Bad points

Degree drops after evaluation can break the equality.

Valid evaluation point

If $h(\eta) \neq 0$, then

$$\mathrm{Res}_{x_1}(f(\eta), g(\eta)) = \mathrm{Res}_{x_1}(f, g)(\eta).$$

Degree Bound
Let

$$D = \deg_{x_1}(f)\deg_{x_2,\ldots,x_n}(g) + \deg_{x_1}(g)\deg_{x_2,\ldots,x_n}(f).$$

Backbone of the Approach

Specialization of the Resultant
Let

$$h = \mathrm{lc}_{x_1}(f)\,\mathrm{lc}_{x_1}(g).$$

Bad points

Degree drops after evaluation can break the equality.

Valid evaluation point

If $h(\eta) \neq 0$, then

$$\mathrm{Res}_{x_1}(f(\eta), g(\eta)) = \mathrm{Res}_{x_1}(f, g)(\eta).$$

Degree Bound
Let

$$D = \deg_{x_1}(f)\deg_{x_2,\ldots,x_n}(g) + \deg_{x_1}(g)\deg_{x_2,\ldots,x_n}(f).$$

Bound

$$\deg\big(\mathrm{Res}_{x_1}(f, g)\big) \leq D.$$

Backbone of the Approach

Specialization of the Resultant
Let

$$h = \mathrm{lc}_{x_1}(f)\,\mathrm{lc}_{x_1}(g).$$

Degree Bound
Let

$$D = \deg_{x_1}(f)\deg_{x_2,\ldots,x_n}(g) + \deg_{x_1}(g)\deg_{x_2,\ldots,x_n}(f).$$

Bad points

Degree drops after evaluation can break the equality.

Valid evaluation point

If $h(\eta) \neq 0$, then

$$\mathrm{Res}_{x_1}(f(\eta), g(\eta)) = \mathrm{Res}_{x_1}(f, g)(\eta).$$

Bound

$$\deg\big(\mathrm{Res}_{x_1}(f, g)\big) \leq D.$$

Interpolation cost

For $n = 2$, at most $D + 1$ evaluation points are needed.

Proposed Bivariate Resultant Algorithm

For $f, g \in \mathbb{Z}[x_1, x_2]$:

$N = \deg_{x_1}(f)\deg_{x_2}(g) + \deg_{x_1}(g)\deg_{x_2}(f)$.

1. Choose $N+1$ good evaluation points.
2. Compute

$$y_i = \mathrm{Res}(f(x_i), g(x_i)).$$

3. Interpolate

$$R = \mathrm{interpolate}(x_i, y_i, N+1).$$

Proposed Bivariate Resultant Algorithm

For $f, g \in \mathbb{Z}[x_1, x_2]$:

$$N = \deg_{x_1}(f)\deg_{x_2}(g) + \deg_{x_1}(g)\deg_{x_2}(f).$$

1. Choose $N+1$ good evaluation points.
2. Compute
$$y_i = \mathrm{Res}(f(x_i), g(x_i)).$$
3. Interpolate
$$R = \mathrm{interpolate}(x_i, y_i, N+1).$$

Implemented strategies:

- Random Points
- Random Positive Points
- Small Random Points
- Small Random Positive Points
- Ordered Points
- Ordered Positive Points

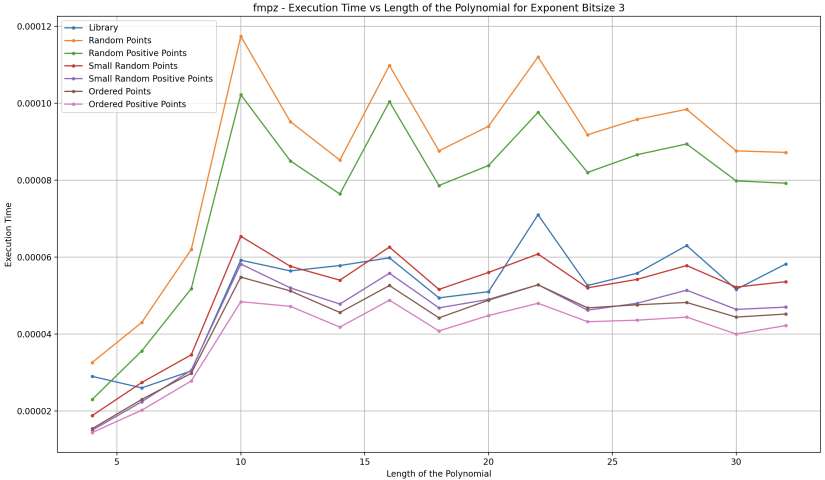
Small and ordered points usually reduce interpolation cost.

FMPZ Benchmarks

Benchmark Setup

- Apple MacBook, 8-core Apple M2 SoC
- 16 GB unified memory
- Average of 5 independent trials
- Coefficient bit size fixed at 50
- Comparison against FLINT bivariate resultant computation

FMPZ Benchmarks



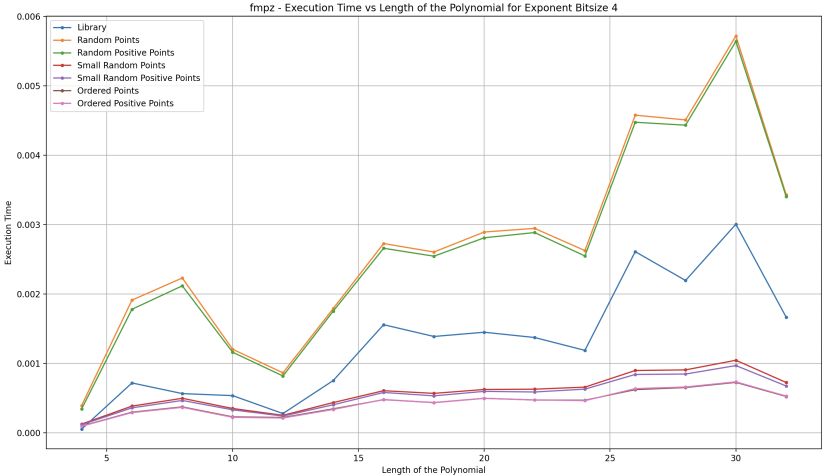
Bitsize 3

Random interpolation points with large bitsize are much slower.

Not too much difference between other benchmarks.

Length of the polynomial does not affect the performance significantly.

FMPZ Benchmarks



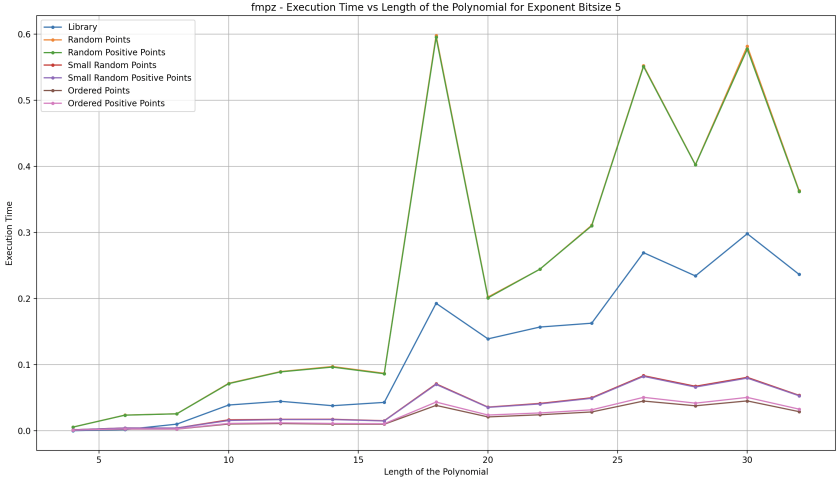
Bitsize 4

Library implementation is slower than our algorithm.

Not too much difference between random interpolation with small bitsize and ordered points.

Execution time increases as the length of the polynomial increases.

FMPZ Benchmarks



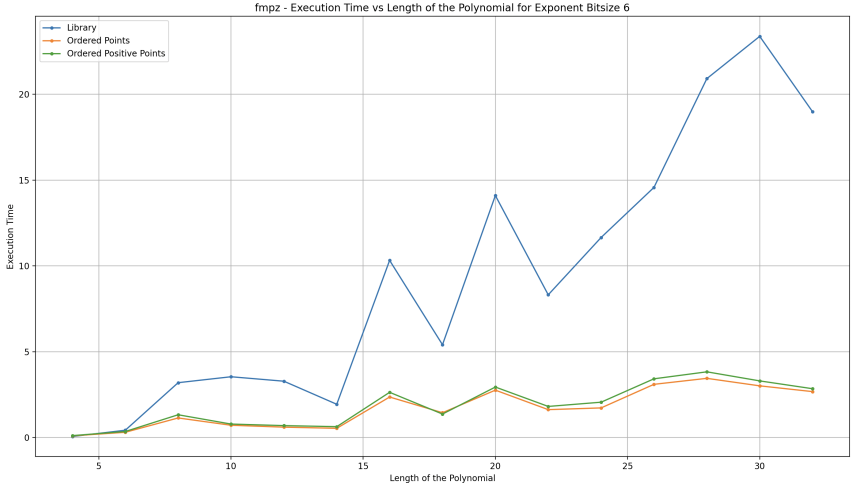
Bitsize 5

Point bitsize strongly affects interpolation time.

We observe high fluctuations in the performance because of random polynomial generation.

Not an accurate measurement of dependance to the length, but the ratio between different methods.

FMPZ Benchmarks



Bitsize 6

We remove random interpolation methods.

We observe 6x performance difference between our methods.

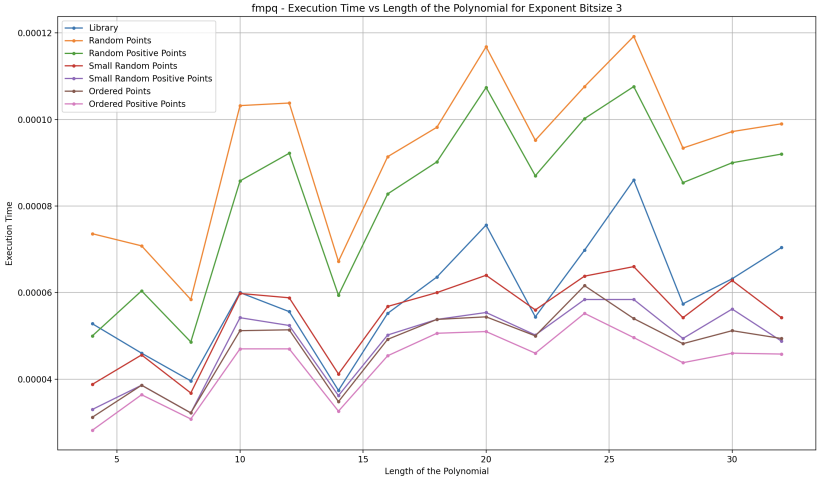
Difference becomes more important as the length of the polynomial & bitsize of the exponent increase.

FMPQ Benchmarks

Benchmark Setup

- Apple MacBook, 8-core Apple M2 SoC
- 16 GB unified memory
- Average of 5 independent trials
- Coefficient bit size fixed at 50
- Comparison against FLINT bivariate resultant computation

FMPQ Benchmarks



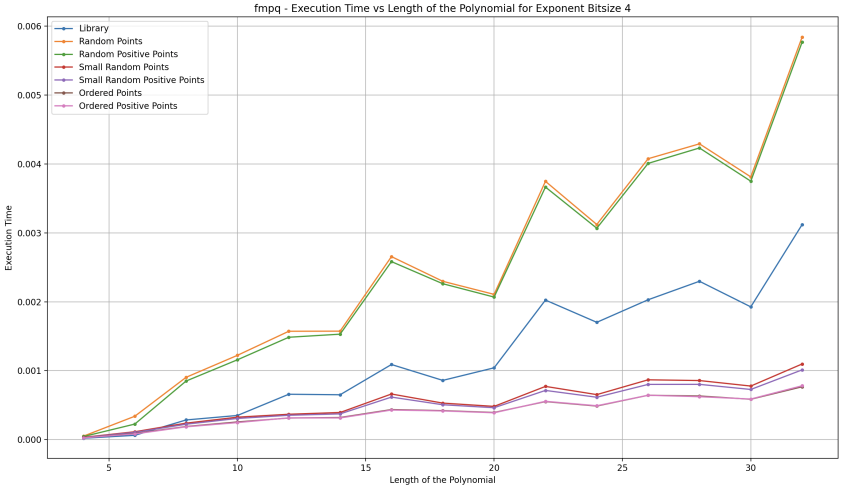
Bitsize 3

Random interpolation points with large bitsize are much slower.

Not too much difference between other benchmarks.

Length of the polynomial does not affect the performance significantly.

FMPQ Benchmarks



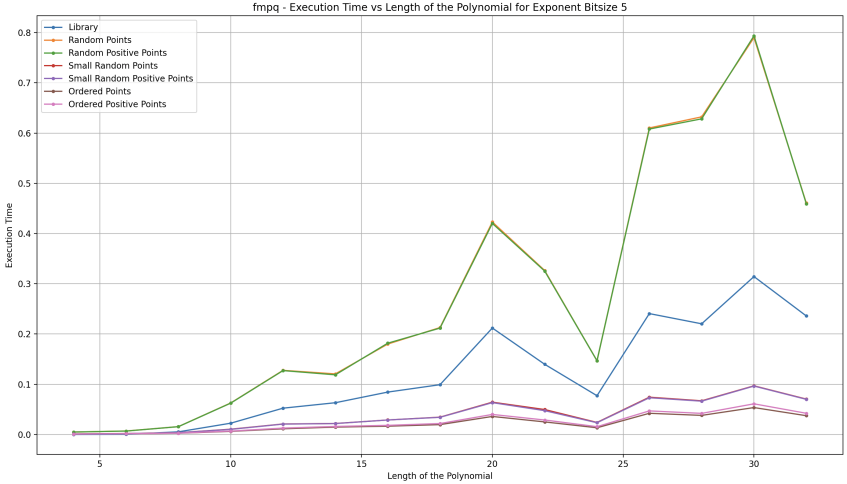
Bitsize 4

Library implementation is slower than our algorithm.

Not too much difference between random interpolation with small bitsize and ordered points.

Execution time increases as the length of the polynomial increases.

FMPQ Benchmarks



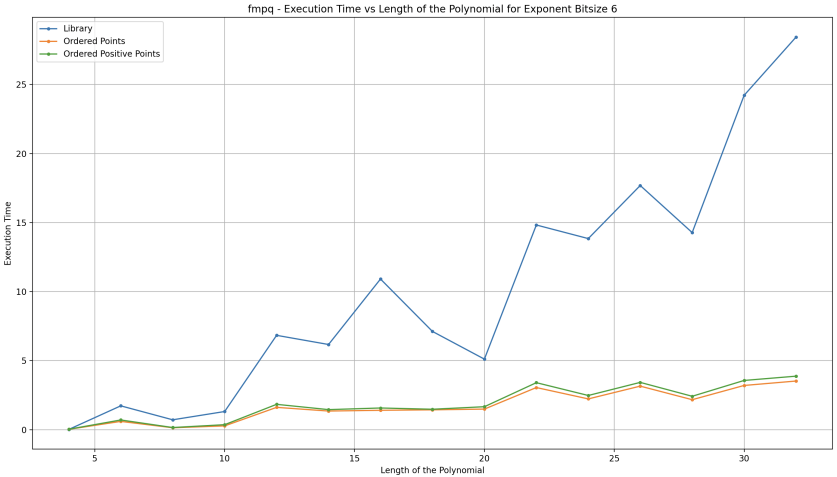
Bitsize 5

Point bitsize strongly affects interpolation time.

We observe high fluctuations in the performance because of random polynomial generation.

Not an accurate measurement of dependance to the length, but the ratio between different methods.

FMPQ Benchmarks



Bitsize 6

We remove random interpolation methods.

We observe 7x performance difference between our methods.

Behaviour is similar to $\mathbb{Z}$ mostly.

Conclusion

Summary of Results

- Subresultants control coefficient growth in GCD/resultant computations.
- Specialization is valid away from leading-coefficient zeros.
- Interpolation recovers multivariate resultants and subresultants.
- Evaluation point choice is critical for performance.
- Ordered or small-bit-size points work best in the benchmarks.
- Our implementation improves FLINT's performance, especially for polynomials with high degree and length.

Conclusion

Summary of Results

- Subresultants control coefficient growth in GCD/resultant computations.
- Specialization is valid away from leading-coefficient zeros.
- Interpolation recovers multivariate resultants and subresultants.
- Evaluation point choice is critical for performance.
- Ordered or small-bit-size points work best in the benchmarks.
- Our implementation improves FLINT's performance, especially for polynomials with high degree and length.

Next Steps

- Extend implementation from resultants / subresultants to subresultant polynomials.
- Extend implementation to higher dimension multivariate polynomials (note that number of points for interpolation increases - $(D+1)^{n-1}$).
- Understand and implement specialization conditions for subresultant polynomials.
- Optimize, test, and benchmark for higher degree and length polynomials.